

Profiling Production Workloads

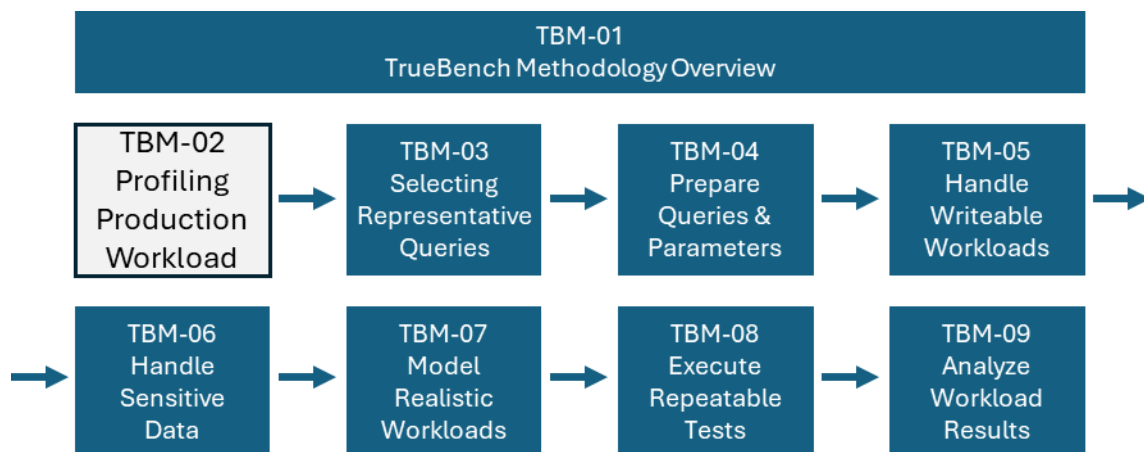
Measure what the system actually does — before testing it.

Author: Douglas H Ebel

Data Warehouse Architect

Business Value and Performance Assessment

Creator of TdBench and TrueBench



Introduction

Profiling the actual workload of an information system is the most important step in validating its performance. **Why invest effort in building a test that does not represent what the system does?**

People responsible for supporting information systems often focus testing on the components that cause them the most operational pain—heavy queries that consume the most resources or large data-loading activities that drive the most support incidents. While these activities are important, they may not represent the most frequent user experience. Without examining actual workload behavior derived from system logs, the resulting tests can easily misrepresent how the system is used.

Over the years, I've seen many tests that ultimately proved very little about how the system is used:

- A U.S. insurance company had tests with **500 and 1,000 identical queries initiated simultaneously** against a single database instance.
- A South American government finance ministry ran tests using only the **worst-performing queries**, many of which were poorly written queries with uncontrolled product joins.
- A telecommunications company tested workloads consisting of **arbitrary 3-, 4-, and 5-table joins** that did not correspond to any real application behavior.
- A large mortgage institution selected production queries but modified them to execute **SELECT COUNT(*)** versions of the queries. The optimizer was able to shortcut execution paths, using a fraction of the original query's resources.

Is this



A good simulation of this?



Each of these tests consumed significant effort, yet none accurately represented how the systems were used.

At first glance, determining tests that represents **hundreds of thousands or even millions of production queries** may appear impossible. Faced with this complexity, teams often resort to shortcuts or intuition. However, a methodical profiling approach makes it practical to construct realistic tests without attempting to reproduce every individual query.

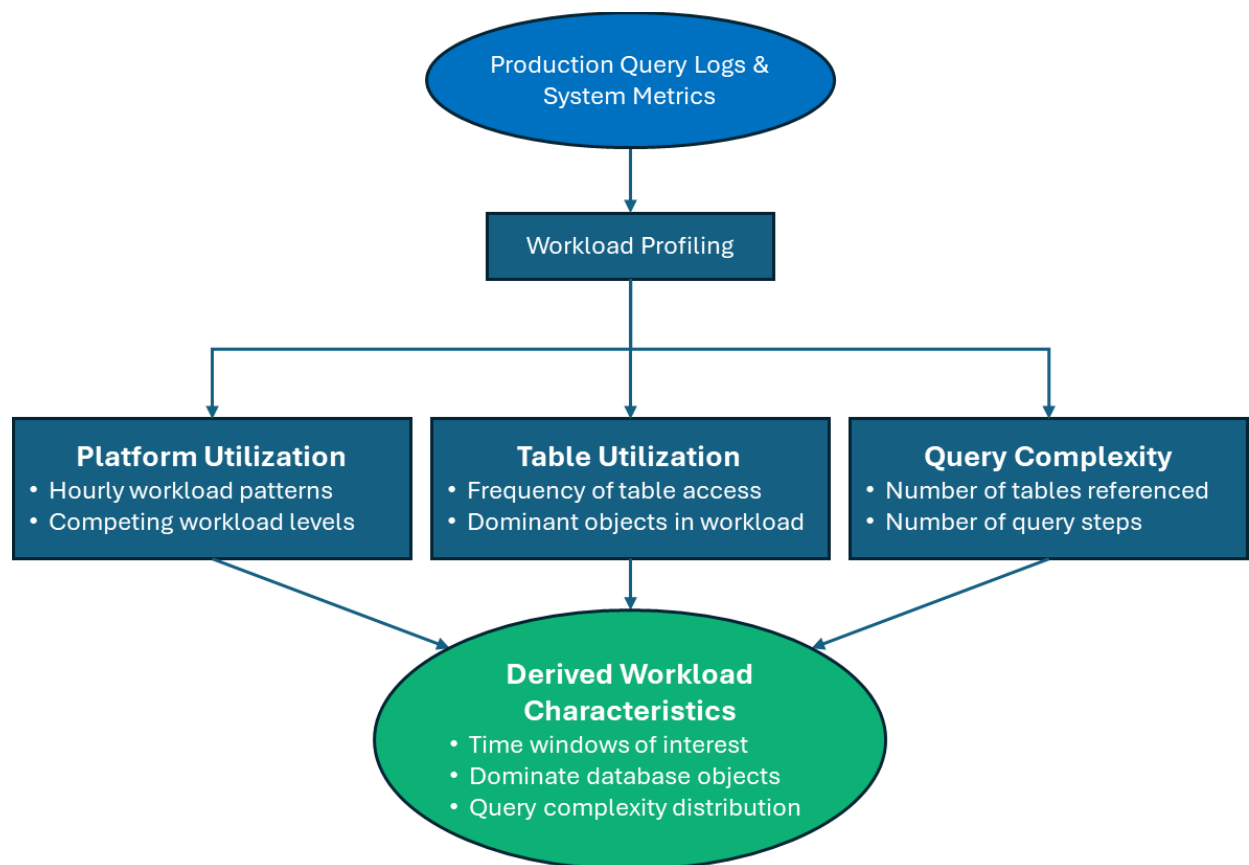
Several key attributes must be examined when profiling a production workload:

- Platform utilization
- referenced tables and views
- query frequency and duration
- query output volume
- query plan characteristics
- concurrency patterns across time periods
- system resource consumption

In many environments, gathering this information may require enabling more detailed logging for a limited period.

The resulting workload profile provides the foundation for the next step in the methodology: **selecting representative queries to replay during validation tests.**

The following diagram illustrates the process:



Platform Utilization

Purpose

Before analyzing individual queries, it is important to understand how the target information system uses the shared computing platform. Most database platforms support multiple applications simultaneously, including production workloads, ad-hoc analysis, and administrative activities.

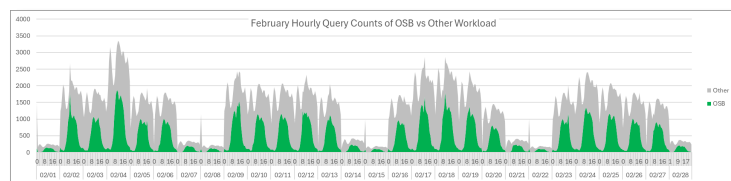
Understanding how the target system fits within overall platform utilization provides essential context for later workload analysis.

How It Is Measured

A useful first step is to examine an hourly summary of platform activity across a representative period, such as a full month. The goal is to compare the activity generated by the target information system with total platform usage.

Workload activity can often be identified through:

- application logon IDs
- database accounts
- query bands
- session attributes embedded by applications



Platform demand can be measured using available system metrics such as CPU seconds, processing credits, or other resource consumption indicators.

The goal at this stage is not detailed query analysis but rather to understand the relative share of system utilization attributable to the workload under study.

Insights Provided

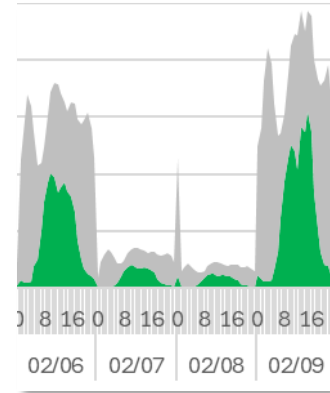
Profiling platform utilization reveals important operating characteristics, including:

- periodic workload activity, such as weekly or monthly processes
- normal daily usage patterns during business hours
- overnight batch processing windows

- time periods when the competing workload is lowest
- potential periods for extracting representative sessions

In many environments, the lowest competing workload occurs during **late night or weekend hours**, which often provide the most practical windows for executing validation tests.

The utilization graph also helps identify periods of interest for extracting sessions that will later be used to define validation workloads.



Detailed examples of how to analyze platform utilization are provided in **Appendix A**.

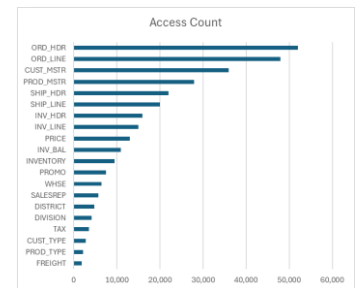
Table Utilization

Purpose

Tables exist to store attributes about business entities and transactions. The problems and opportunities that drive the business determine how frequently those tables are used.

A realistic validation test must reflect **which tables are accessed most often**, not just the structure of individual queries.

It is also important to determine whether the workload includes **updates to tables**. If updates are part of the workload, the design of the validation test must ensure that production data is not affected and that tables can be reset between test executions.



How It Is Measured

Table utilization is measured by analyzing how frequently tables are referenced by queries over a representative period, such as a full month.

On platforms such as Teradata, this information can be obtained from system logging. For example, the DBC.QryLogObjects view records table references associated with each query.

By aggregating these references, it is possible to determine how often each table is accessed for:

- read activity
- update activity

On platforms without object-level logging, table references can be approximated by analyzing SQL text to identify objects used in FROM, JOIN, INSERT, and UPDATE clauses.

The objective is to produce a **frequency distribution of table access across the workload**.

Insights Provided

Profiling table utilization reveals:

- which tables dominate read activity
- which tables are frequently updated
- which tables have minimal impact on the workload

These observations define the **data footprint of the workload** and indicate whether update activity must be included in validation testing.

If update activity is required, the design of the validation test becomes more complex:

- tables targeted by updates may need to be copied or isolated to allow repeatable testing
- some applications provide “dummy” customers or transactions that can be safely used for update activity
- test logons must be carefully controlled to ensure that validation tests do not modify production data

These factors can significantly influence the design of the validation environment and are addressed in later sections.

This information is later used to ensure that selected queries reflect the same pattern of table access observed in production.

Detailed examples of table utilization analysis are provided in **Appendix B**.

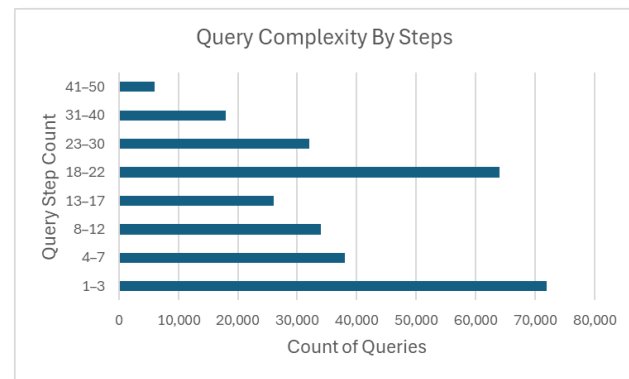
Query Complexity

Purpose

After understanding when the workload runs and which tables it uses, the next step is to understand **how complex the queries are**.

Some information systems expose users to semantic views that combine multiple normalized tables into business facts that are easier to query. A user may think a query is simple because it references only one or two views, while the DBMS may actually expand those views into a much larger number of underlying tables and joins.

In other cases, multiple semantic views may be combined in a single query, sometimes reusing the same underlying tables. Database optimizers may also break a single join into multiple execution steps or apply different strategies when data distributions are skewed.



A realistic validation test must reflect the **same general level of query complexity** as observed in the production workload.

How It Is Measured

Query complexity can be measured using characteristics available from query logging or derived from query text over a representative period, such as a full month.

On platforms such as Teradata, the query log provides useful indicators, including:

- query step count
- number of tables referenced

These two measures are not the same. Queries with the same number of referenced tables may produce different numbers of steps depending on view expansion, optimizer strategy, data distribution, and query structure.

On platforms with less detailed logging, approximate complexity can be derived by analyzing SQL text to determine the number of referenced tables and by classifying common query patterns.

The objective is to produce a **distribution of query complexity across the workload**, rather than focusing only on the most complex queries.

Insights Provided

Profiling query complexity reveals:

- the proportion of simple versus complex queries in the workload
- whether the workload is dominated by short tactical queries, larger analytical joins, or a mix of both
- the extent to which semantic views hide underlying query complexity
- whether a small number of highly complex queries account for a disproportionate share of resource demand

These observations help define the **structural complexity of the workload** and provide guidance when selecting queries for validation testing. The selected workload for validation testing should reflect similar complexity characteristics to those observed in production.

Detailed examples of query complexity analysis are provided in **Appendix C**.

Key Takeaways

- profiling the production workload is essential to understanding how the system is actually used
- platform utilization identifies when the workload runs and when competing activity is lowest
- table utilization reveals which database objects dominate workload activity and whether updates must be included in testing
- query complexity shows how simple and complex queries are distributed across the workload
- together, these dimensions—when the workload runs, which tables it uses, and how complex the queries are—define the structure of the production workload

This structure provides the foundation for selecting representative queries for validation testing.

Appendix A —Platform Utilization Example

The following example summarizes hourly workload activity for one month, separating the target information system from other platform activity. In this example, usernames that begin with "osb" identify the target system.

The result set can be copied into Excel to create a graph showing:

- hourly query count for the target system versus other workloads
- hourly CPU demand for the target system versus other workloads

These graphs can be used to identify:

- periods of reduced competing workload, such as late nights or weekend hours

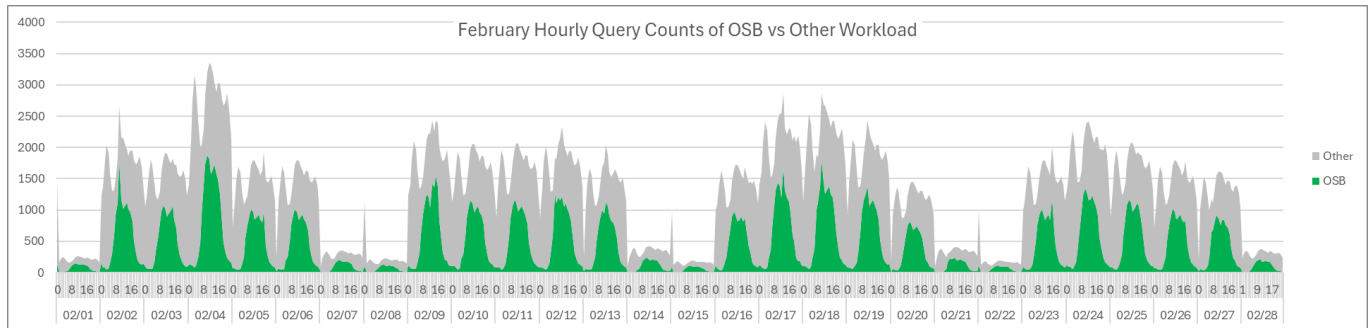
Because date and time functions vary across database platforms, it is often simplest to derive additional calendar attributes, such as day of week, in Excel after exporting the query results.

The following example is for Teradata, but equivalent queries can be written for other platforms:

```
SELECT
    CASE
        WHEN username LIKE 'osb%' THEN 'OSB'
        ELSE 'other'
    END AS UserType,
    CAST(starttime AS DATE) AS TheDate,
    EXTRACT(HOUR FROM starttime) AS TheHour,
    COUNT(*) AS QueryCount,
    SUM(ampcputime) AS CpuSecs
FROM dbc.qrylog
WHERE TheDate >= '2026-02-01'
    AND TheDate < '2026-03-01'
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

In this example, a logon ID pattern is used to distinguish the target system from other system activities. In practice, other identifiers such as account strings or query bands may be used. You may also be able to separate sessions that perform data maintenance from those that perform user query activity. However, too many “user types” will make the viewing of results more difficult.

Below is a sample of the output:



A similar output can be produced for CPU or other platform utilization metrics.

This output is used to identify periods for selecting candidate sessions for validation testing.

Appendix B — Table Utilization Example

The following example illustrates how to determine **which tables are accessed most frequently** by a production workload over a representative period.

The objective is to produce a **frequency distribution of table access** that identifies the workload's dominant tables and distinguishes between read and write activity. This information directly impacts how validation tests must be constructed, particularly when update activity is present.

When sessions are selected for testing, the distribution of selected sessions should both represent heavily used tables and cover lesser-used tables.

Example Using Teradata

On Teradata, object-level access is available directly through system logging. The DBC.QryLogObjects view records the tables referenced by each query.

The following query aggregates table access across one month:

```
WITH RawUsage AS
(
    SELECT
        QueryID,
        ObjectDatabaseName,
        ObjectTableName,
        TargetIndicator
    FROM dbc.qrylogobjects
    WHERE objectType = 'Tab'
        AND CAST(CollectTimeStamp AS DATE)
            BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'
    GROUP BY 1,2,3,4
)
SELECT
    ObjectDatabaseName,
    ObjectTableName,
    TargetIndicator AS WriteMode,
    COUNT(DISTINCT QueryID) AS QueryCount
FROM RawUsage
GROUP BY ObjectDatabaseName, ObjectTableName, WriteMode
ORDER BY QueryCount DESC;
```

The TargetIndicator column identifies whether the table is the target of an update:

- 'Y' — table is written or updated
- 'N' — table is read

This produces a list of tables ordered by access frequency, along with an indication of update activity.

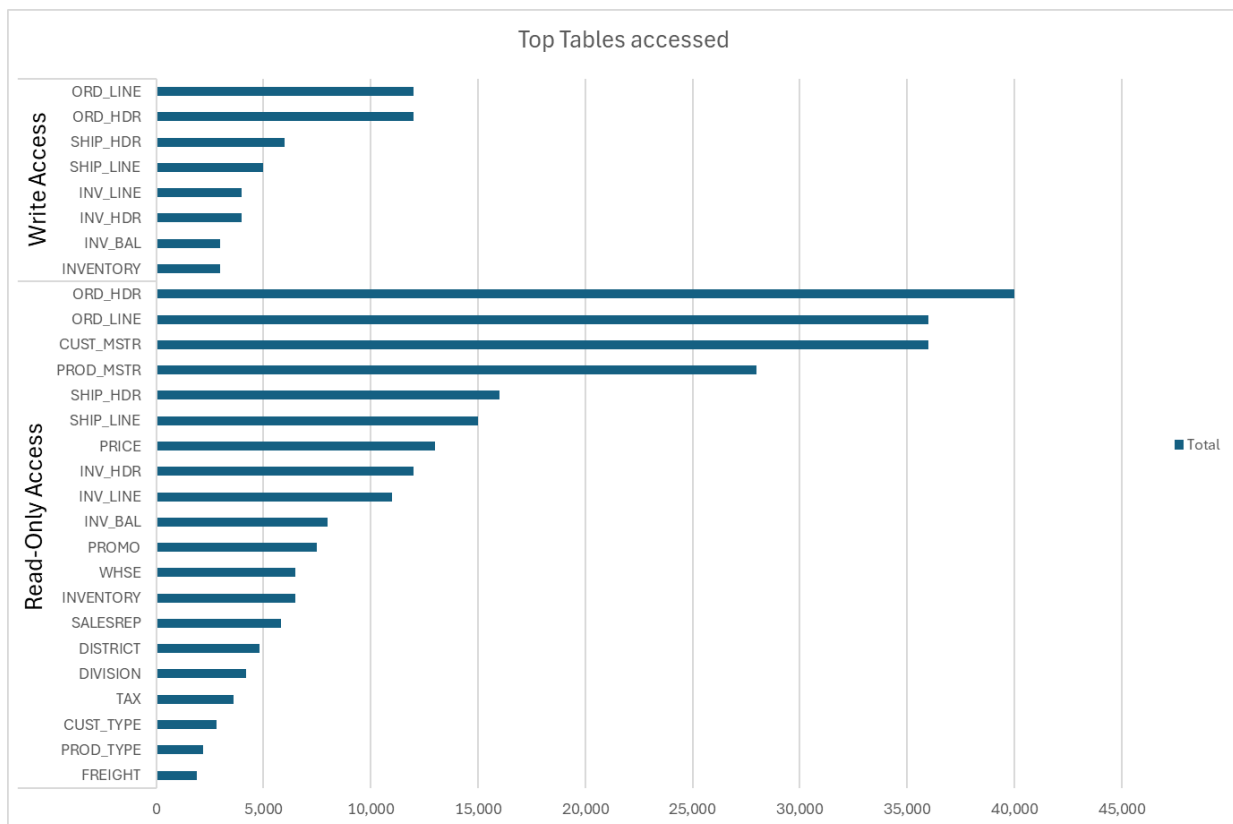
Interpreting the Results

The output can be used to create a chart showing:

- the most frequently accessed tables in the workload
- the relative distribution of table access
- which tables are involved in update activity

In most environments, a relatively small number of tables account for the majority of query activity, while many tables are accessed infrequently.

This distribution defines the **data footprint of the workload**. It also identifies whether validation testing can be read-only or must safely accommodate updates.



Platforms Without Object-Level Logging

Some database platforms do not provide direct access to table-level usage. In these cases, table access can be approximated by analyzing SQL text from query history.

A practical approximation approach is:

- identify the statement type (SELECT, INSERT, UPDATE, DELETE)
- extract table names referenced in FROM, JOIN, UPDATE, and INSERT INTO clauses
- normalize object names
- aggregate counts across the workload

Simple pattern-matching techniques, such as regular expressions, can be used to extract table references. For example:

FROM <object>

JOIN <object>

UPDATE <object>

INSERT INTO <object>

This approach provides a reasonable approximation of table usage without requiring complex parsing.

Limitations of SQL-Based Extraction

When deriving table usage from SQL text, several limitations should be considered:

- Tables referenced using non-ANSI joins (comma-separated lists with WHERE conditions) may not be fully captured
- Semantic views may hide underlying tables, causing the analysis to reflect view usage rather than base tables
- Nested queries and common table expressions may require additional parsing logic
- Dynamically generated SQL may introduce inconsistencies

Because of these limitations, the results should be viewed as an **approximation of table usage** rather than an exact reconstruction.

Practical Guidance

The objective of this analysis is not to identify every table reference, but to determine:

- which tables dominate the workload
- whether the update activity is a significant component
- whether the workload is concentrated on a small set of objects

Even approximate methods are sufficient to identify the dominant tables, which is the primary requirement for defining representative validation workloads.

Table utilization analysis provides a practical way to understand the data footprint of a production workload.

When combined with platform utilization and query complexity, this information helps define the workload structure and supports the selection of representative queries for validation testing.

Appendix C — Query Complexity Example

This appendix illustrates how to characterize the **complexity of queries** within a production workload.

The objective is to understand how queries vary in terms of:

- execution complexity (number of processing steps)
- structural complexity (number of tables referenced)
- result size (volume of data returned)

Together, these dimensions describe how demanding queries are on the database, storage, and network infrastructure.

Interpreting the Results

In most workloads, query complexity follows recognizable patterns:

- a large number of simple queries with few steps and small result sets
- a smaller number of moderately complex queries, often driven by semantic views
- a limited number of highly complex queries that consume a disproportionate share of resources

Result size provides an additional dimension of behavior:

- Some queries are computationally simple but return large result sets
- Large result sets can place a significant demand on network bandwidth and client processing
- User-perceived performance is often influenced as much by data transfer as by execution time

A practical way to interpret result size is to group queries by logarithmic ranges:

Lines	LogRowsOut
1	0
10	1
100	2
1,000	3
10,000	4
100,000	5
1,000,000	6

These ranges help identify whether the workload includes a significant proportion of large-result queries.

Example Using Teradata

On Teradata, query complexity can be derived from system logging tables.

The following query summarizes query characteristics across a representative period:

```
WITH RawComplexity AS
(
  SELECT
    ql.QueryID,
    ql.NumSteps,
    CAST(LOG(NULLIFZERO(ql.NumResultRows)) AS INTEGER) AS LogRowsOut,
    COUNT(DISTINCT qlo.ObjectDatabaseName || qlo.ObjectTableName) AS
    TableCount
  FROM dbc.qrylog ql
  JOIN dbc.qrylogobjects qlo
    ON ql.QueryID = qlo.QueryID
  WHERE qlo.ObjectType = 'Tab'
    AND CAST(ql.CollectTimeStamp AS DATE)
      BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'
    AND CAST(qlo.CollectTimeStamp AS DATE)
      BETWEEN DATE '2026-02-01' AND DATE '2026-02-28'
  GROUP BY 1, 2, 3
)
SELECT
  NumSteps,
  TableCount,
  LogRowsOut,
  COUNT(*) AS QueryCount
FROM RawComplexity
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

This query produces a distribution of queries by:

- number of execution steps (NumSteps)
- number of tables referenced (TableCount)
- logarithmic scale of rows returned (LogRowsOut)

Example: Step Range vs Result Size Distribution

Complexity (Steps)	Log (Result Rows)						Grand Total
	0	1	2	3	4	5	
1-3	39,581	22,973	8,294	1,542	400	103	72,893
4-7	13,390	11,879	8,165	2,943	761	198	37,336
8-12	5,743	8,100	10,192	6,786	2,585	475	33,881
13-17	2,866	4,989	8,243	5,715	3,133	981	25,927
18-22	5,245	8,573	14,353	17,683	11,482	4,810	62,146
23-30	1,900	4,986	8,017	9,824	4,871	2,015	31,613
31-40	960	2,094	5,148	4,986	3,514	1,474	18,176
41-50	512	959	1,529	1,534	991	491	6,016
Grand Total	70,197	64,553	63,941	51,013	27,737	10,547	287,988

Most queries in the workload are simple, returning fewer than 10 rows with minimal execution steps. These are typically application lookups, validations, or metadata queries that contribute heavily to query counts but consume relatively few resources.

A second concentration appears in the **18–22 step range**, where queries return moderate to large result sets. These are often generated by BI tools or semantic views that combine multiple tables into business-level queries.

A smaller number of higher-step queries (23+ steps) produce disproportionately large result sets, representing analytical or reporting workloads that place greater demand on both the database and network.

Identifying Noise Queries

Production workloads often include many simple queries generated by tools and applications, such as:

- metadata lookups against system catalogs
- session initialization queries
- small validation or existence checks

These queries may significantly increase query counts without representing meaningful business activity.

Depending on the objective of the analysis, it may be useful to:

- identify and optionally exclude these queries from the complexity distribution
- ensure that the selected workload reflects actual business usage patterns

Platforms Without Detailed Logging

Not all database platforms provide detailed metrics such as step counts or result row counts.

In these environments, query complexity can be approximated using SQL text analysis:

- count the number of tables referenced in FROM and JOIN clauses
- identify query type (SELECT, INSERT, UPDATE, DELETE)
- use query execution time as a proxy for complexity

While less precise, these methods are generally sufficient to identify relative differences between simple and complex queries.

Practical Guidance

The objective of this analysis is not to measure every aspect of query behavior, but to understand the **distribution of workload complexity**.

In particular, the analysis should identify:

- the balance between simple and complex queries
- whether a small number of complex queries dominate resource consumption
- whether large result sets are a significant component of the workload

Validation workloads should reflect these characteristics to ensure that test results are representative of production behavior.

Query complexity analysis provides insight into how work is distributed across simple, moderate, and complex queries, as well as how much data is returned to users and applications.

When combined with platform utilization and table utilization, this analysis completes the picture of how the production workload behaves and supports the selection of representative queries for validation testing.